

SC4053 Blockchain Technology
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-30 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

1	Blockchain Foundations and Hash Chains	1
1.1	Why Not Just a Database?	1
1.2	Hash Functions and the Chain	2
1.3	Anatomy of a Block	4
1.4	Exercises	4
2	Consensus: Proof-of-Work, Proof-of-Stake and BFT	6
2.1	The Consensus Problem	6
2.2	Proof-of-Work: The Mining Lottery	7
2.3	Proof-of-Stake: Lottery by Stake	8
2.4	Byzantine Fault Tolerance (BFT)	8
2.5	Difficulty Adjustment and Fork Choice	9
2.6	Exercises	10
3	Transactions, UTXO and the Account Model	11
3.1	Transactions as Value Flow	11
3.2	The UTXO Model	12
3.3	The Account Model	13
3.4	Mempool, Fees and Block Packing	14
3.5	Exercises	14
4	Smart Contracts: Solidity and the EVM	16
4.1	From Vending Machines to On-Chain Programs	16
4.2	The EVM Execution Model	17

4.2.1	Calls and delegatecall	17
4.3	Writing Solidity	17
4.3.1	Errors that cost millions	19
4.4	Exercises	19
5	Cryptography: Signatures, Merkle Trees and Commitments	21
5.1	Digital Signatures: Seals that Mathematics Checks	21
5.2	Merkle Trees: One Hash Commits to Many	22
5.3	Commitments and Zero-Knowledge Intuition	23
5.4	Exercises	24
6	Scaling: Channels, Rollups and Sharding	25
6.1	Why Layer-1 Cannot Scale Alone	25
6.2	Payment Channels and Lightning	26
6.3	Rollups: Execute Off-Chain, Prove On-Chain	27
6.4	Sharding and Data Availability	28
6.5	Exercises	28
7	Security and Attacks: Double-Spend, MEV and Verification	30
7.1	Double-Spend and Fork Races	30
7.2	MEV and Front-Running	31
7.3	Auditing and Formal Verification	32
7.4	Exercises	33
8	Applications, Tokenomics and Governance	35
8.1	Application Patterns	35
8.2	Tokenomics: Designing Incentives	36
8.3	Governance, DAOs and Trade-offs	37
8.4	Exercises	38

Chapter 1

Blockchain Foundations and Hash Chains

“A blockchain is a chain of blocks where each block points to the previous one with a fingerprint. Break one link and every later fingerprint changes.” — This chapter builds that chain from scratch.

From zero: no blockchain background assumed. We start from a notebook where each page contains a fingerprint of the previous page. We define hashes, linked lists and content addressing from first principles, then assemble them into a tamper-evident chain. Every term is defined on first use. We move from intuition to formal guarantees to code you can run.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain what makes a blockchain different from a database or a Git log;
- (L2) define cryptographic hash functions and their three security properties;
- (L3) construct and verify a hash chain and prove tamper evidence;
- (L4) describe a block (header, body, Merkle root, nonce) and its serialisation;
- (L5) argue why decentralisation is about trust, not just replication.

1.1 Why Not Just a Database?

Imagine a shared notebook for a club treasury. Anyone can write an entry, anyone can read it, and the notebook is photocopied to every member. How do we know no one secretly rewrote page 3 to give themselves extra money? A database solves this with a trusted administrator who controls writes. A blockchain solves it without one.

Three ideas combine. First, *content addressing*: each page stores a fingerprint (hash) of the previous page, so pages are linked by content, not by page number. Second, *replication*: every participant holds a full copy.

Third, *verification*: anyone can recompute fingerprints and check the chain. Together they give *tamper evidence*: changing history is not prevented by access control but made detectable by mathematics.

We contrast three architectures:

Centralised

One server decides. Fast, simple, but requires trust in the operator.

Decentralised

Many independent operators hold copies; no single party controls the log.

Distributed but trusted

Google Docs replicates for performance, but Google still owns the truth.

Blockchain targets the second case: many mutually distrusting parties need one shared truth.

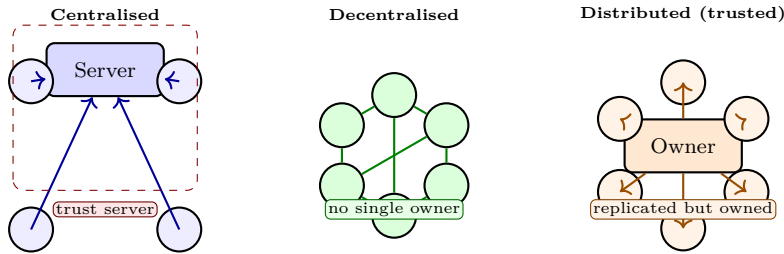


Figure 1.1: Three replication stories. Only the middle gives every node equal authority. Blockchain adds hash-linking (next figure) so copies can be cross-checked without trusting any one replica. Colour groups trust (blue central, green decentralised, orange owner-replicated).

1.2 Hash Functions and the Chain

A *hash function* $H : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$ maps any string to a fixed 256-bit fingerprint. We need:

Definition 1.1 (Hash security). H is (i) *preimage resistant*: given y , hard to find x with $H(x) = y$; (ii) *second-preimage resistant*: given x , hard to find $x' \neq x$ with $H(x') = H(x)$; (iii) *collision resistant*: hard to find any $x \neq x'$ with $H(x) = H(x')$. It is also *avalanche*: flipping one bit of x changes $\approx 50\%$ of output bits.

In practice we use SHA-256. Treat it as a random oracle for reasoning.

A *hash chain* links items $B_0, \dots, B_n$ by storing inside B_i the hash of B_{i-1} :

$$h_i = H(B_i \parallel h_{i-1}), \quad h_{-1} := 0^{256}, \quad \text{chain head} = h_n. \quad (1.1)$$

To verify, recompute $h_0, \dots, h_n$ and compare the final h_n . Tampering with any B_k changes h_k (avalanche) and thus every later $h_{k+1}, \dots, h_n$ – a domino effect.

Theorem 1.1 (Tamper evidence). If H is collision resistant, any change to B_k with $k \leq n$ is detected by recomputing the chain unless the adversary finds a collision in H .

Proof sketch. Changing B_k changes its hash with overwhelming probability (second-preimage resistance), so recomputed $h_k \neq$ stored h_k in B_{k+1} , and inductively all later headers mismatch. $\square$

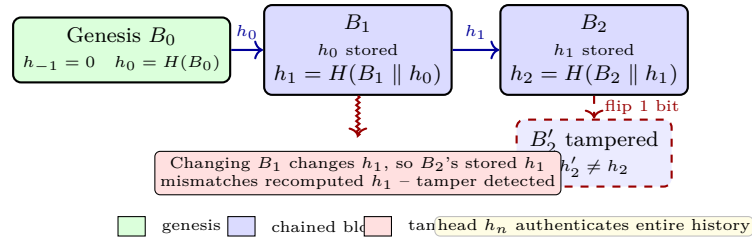


Figure 1.2: Hash chain: each block commits to all previous blocks via nested hashes. Below, tampering with B_1 ripples forward (red zigzag). Only the head hash h_n needs to be remembered to verify everything.

Example 1.1 (Git vs blockchain). Git commits are hash-chained (each commit hashes its parent). The difference is authority: Git allows history rewriting via force-push; a blockchain's consensus rules forbid rewriting without redoing proof-of-work or acquiring stake, which we formalise in Ch. 2.

A runnable hash chain in Python makes this concrete.

```

1 import hashlib, json
2
3 def H(x: bytes) -> str:
4     return hashlib.sha256(x).hexdigest()
5
6 def hash_block(data, prev_hash):
7     payload = json.dumps(data, sort_keys=True).encode() + prev_hash.encode()
8     return H(payload)
9
10 # Build chain B0 -> B1 -> B2
11 chain = []
12 prev = "0"*64
13 for data in [{"tx": "Alice->Bob 10"}, {"tx": "Bob->Carol 5"}, {"tx": "Carol->Dave 2"}]:
14     h = hash_block(data, prev)
15     chain.append((data, prev, h))
16     prev = h
17     print(data, "prev=", prev[:10]+"...", "hash=", h[:12]+"...")
18
19 # Verify
20 def verify(chain):
21     prev = "0"*64
22     for data, stored_prev, stored_h in chain:
23         assert stored_prev == prev, "prev mismatch -- tamper!"
24         assert stored_h == hash_block(data, prev), "hash mismatch!"
25         prev = stored_h
26     return True
27
28 print("verify:", verify(chain))
29 # Tamper: flip one character in B1
30 chain[1] = ({ "tx": "Bob->Carol 5000" }, chain[1][1], chain[1][2])
31 try: verify(chain)
32 except AssertionError as e: print("tamper detected:", e)

```

1.3 Anatomy of a Block

A block bundles many transactions for efficiency. Splitting *header* (small, hash-linked) from *body* (large, Merkle-committed) lets light clients verify inclusion with $O(\log n)$ hashes (Ch. 5).

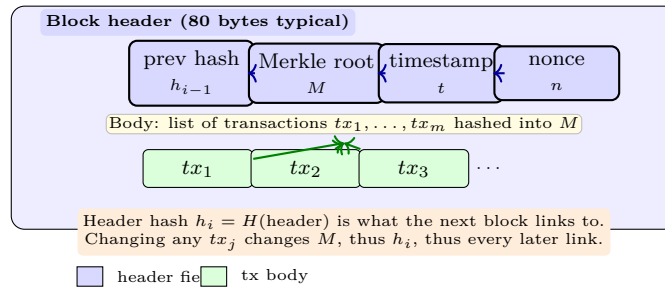


Figure 1.3: Block anatomy: a tiny header commits to a large body via the Merkle root. Light clients store only headers yet can verify any transaction with a Merkle proof.

```

1 import hashlib, struct, time
2 def sha256d(b: bytes) -> bytes:
3     return hashlib.sha256(hashlib.sha256(b).digest()).digest()
4 # Simplified header serialisation (Bitcoin-like)
5 def header_bytes(prev_hash_hex, merkle_hex, timestamp, nonce):
6     return bytes.fromhex(prev_hash_hex)[::-1] + bytes.fromhex(merkle_hex)[::-1] \
7         + struct.pack("<II", timestamp, nonce)
8 prev = "00"*32
9 merkle = hashlib.sha256(b"tx1|tx2|tx3").hexdigest()
10 hdr = header_bytes(prev, merkle, int(time.time()), 42)
11 print("header hash:", sha256d(hdr)[::-1].hex()[:16]+"...")

```

1.4 Exercises

- E1.1 **Avalanche drill.** Take any sentence, compute SHA-256, flip one bit, recompute. Count differing bits. Repeat 5 times and report the distribution. Why does this matter for tamper evidence?
- E1.2 **Chain surgery.** Given a 4-block chain, you tamper B_2 . Which stored hashes become invalid? Must you recompute only h_2 or all of $h_2, \dots, h_4$ to hide the change? Relate to collision resistance.
- E1.3 **Git vs chain.** Explain in one paragraph how Git's hash chain differs from a blockchain's regarding authority to rewrite history. What would Git need to become a blockchain?
- E1.4 **Header/body split.** A light client stores only 80-byte headers for a chain of 10^6 blocks. Estimate storage vs a full node with 1 MB bodies. How does a Merkle proof let the light client verify a transaction without the body?
- E1.5 **Trust without trust.** A naive design stores "page number" links instead of hash links. Show an explicit tampering that page numbers miss but hash links catch. Generalise to why content addressing is essential for decentralisation.

Chapter Notes

Nakamoto's Bitcoin whitepaper (2008) introduced the hash chain plus proof-of-work. For hash functions see Katz–Lindell, *Introduction to Modern Cryptography*, Ch. 5. Merkle trees (1979) are detailed in Ch. 5. Narayanan et al., *Bitcoin and Cryptocurrency Technologies* (Princeton, 2016) gives the gentlest full-stack introduction; Buterin's Ethereum whitepaper (2013) extends the model to state machines.

Chapter 2

Consensus: Proof-of-Work, Proof-of-Stake and BFT

“Consensus is how strangers agree on one history when anyone can propose the next page.” — From mining puzzles to validator lotteries to Byzantine voting, this chapter derives why agreement is hard and how blockchains achieve it.

From zero: no consensus background assumed. We start from a classroom where every student shouts a different answer - how do you pick one? We build intuition with lotteries, then formalise safety/liveness/finality, and compare PoW, PoS and BFT step by step with runnable lotteries.

Learning Outcomes

After this chapter you will be able to:

- (L1) state safety, liveness and finality for a blockchain and why FLP matters;
- (L2) derive PoW success probability, difficulty adjustment and expected block time;
- (L3) explain PoS selection, slashing and the nothing-at-stake problem;
- (L4) trace PBFT’s three phases and the $n \geq 3f + 1$ bound;
- (L5) compare PoW/PoS/BFT on energy, throughput and adversary thresholds.

2.1 The Consensus Problem

A blockchain is a replicated log: every honest node should eventually hold the same sequence $B_0, B_1, \dots$. In SC4051 terms this is total-order broadcast, but now participants are *open* (anyone can join) and *Byzantine* (some may lie for profit). Classical consensus assumed a fixed n ; blockchain consensus must work with unknown, changing n and economic incentives.

Definition 2.1 (Safety, liveness, finality). *Safety*: no two honest nodes commit different blocks at the same height. *Liveness*: new honest transactions are eventually committed. *Finality* is the guarantee that a committed block will never be reverted – *probabilistic* (reversion probability decays with depth, as in Bitcoin) vs *deterministic* (once committed, never reverted, as in PBFT).

The FLP impossibility (Fischer–Lynch–Paterson) says deterministic consensus is impossible in fully asynchronous networks with one crash. Blockchains escape FLP by adding randomness (PoW lottery), timing assumptions (partial synchrony for BFT), or economic stakes (PoS).

2.2 Proof-of-Work: The Mining Lottery

PoW turns block proposal into a computational lottery. To propose B_i , a miner repeatedly picks a nonce n and checks

$$H(\text{header}(B_i) \parallel n) < T, \quad T = \frac{2^{256}}{D}, \quad (2.1)$$

where D is *difficulty*. The smaller T , the harder the puzzle. Because H is pseudorandom, each try succeeds with probability $p = T/2^{256} = 1/D$, independent of history.

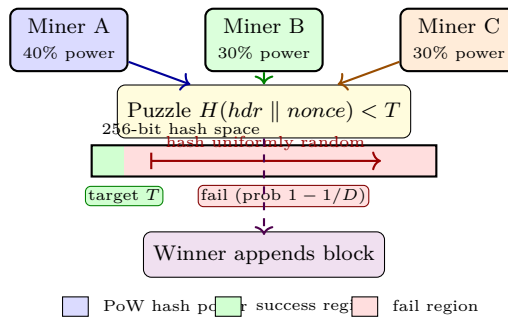


Figure 2.1: PoW as a parallel lottery: each hash is an independent ticket, uniform in $[0, 2^{256})$. Success lands in the tiny green target T ; difficulty D sets $T = 2^{256}/D$. Whoever hits T first wins the block.

If the global hash rate is r hashes/s, the network finds a block as a Poisson process with rate $\lambda = r/D$, so expected block time $\mathbf{E}[T_{\text{block}}] = D/r$. Bitcoin retargets D every 2016 blocks to keep $\mathbf{E}[T_{\text{block}}] \approx 10$ min.

```

1 import hashlib, time, random
2 def sha256d(b): return hashlib.sha256(hashlib.sha256(b).digest()).hexdigest()
3 def mine(header_prefix, target_hex, max_nonce=5000000):
4     T = int(target_hex, 16)
5     for nonce in range(max_nonce):
6         h = hashlib.sha256((header_prefix+str(nonce)).encode()).hexdigest()
7         if int(h,16) < T:
8             return nonce, h
9     return None
10 # Demo: difficulty with 3 leading hex zeros (12 bits)
11 target = "000"+"f"*61 # ~ D = 16^3 = 4096
12 nonce, h = mine("block42|prev=abc|merkle=def|", target)
13 print(f"found nonce {nonce} hash {h[:16]}... < target")
14 # Expected tries ~ D = 4096, ~ milliseconds on a laptop; real Bitcoin D ~ 10^13x larger

```

Remark 2.1 (Energy and incentives). PoW ties security to energy: rewriting k blocks costs $\approx k \cdot D$ hashes. An

attacker with $< 50\%$ power succeeds with probability that decays exponentially in k (Nakamoto's analysis). The cost is electricity; the reward is block subsidy plus fees. When reward $<$ cost, miners leave and security drops – a coupling PoS tries to break.

2.3 Proof-of-Stake: Lottery by Stake

PoS replaces energy with escrowed capital. Validators lock (*stake*) coins; the protocol pseudo-randomly picks the next proposer with probability proportional to stake. No hashing race – a single hash plus signature suffices.

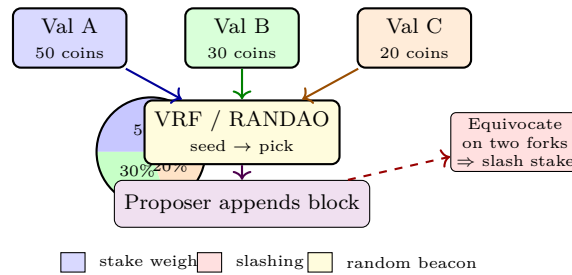


Figure 2.2: PoS proposer lottery weighted by stake, plus slashing (red) that punishes double-signing. Comparison: PoW weighs by hash power; PoS weighs by escrowed coins. Legend distinguishes power, randomness and punishment.

The *nothing-at-stake* worry: without cost to vote on multiple forks, validators might rationally double-sign. Modern PoS (Ethereum's Gasper) adds *slashing* (burn stake if you sign conflicting blocks), *finality gadgets* (Casper FFG) and *weak subjectivity* checkpoints so late joiners know the canonical chain.

```

1 import random, hashlib
2 def pos_lottery(stakes, seed="beacon42"):
3     # stakes: dict name -> coins; returns winner
4     h = int(hashlib.sha256(seed.encode()).hexdigest(), 16)
5     total = sum(stakes.values())
6     pick = h % total
7     cur = 0
8     for name, s in stakes.items():
9         cur += s
10        if pick < cur: return name
11 stakes = {"Alice":50, "Bob":30, "Carol":20}
12 for seed in ["b0", "b1", "b2", "b3", "b4"]:
13     print(seed, "->", pos_lottery(stakes, seed))
14 # Over many beacons, win rate -> 50/30/20%
```

2.4 Byzantine Fault Tolerance (BFT)

When the validator set is known and small ($n \approx 100$), classical BFT shines. PBFT (Castro–Liskov) achieves deterministic finality with $n \geq 3f + 1$ under partial synchrony.

Theorem 2.1 (BFT lower bound). No deterministic BFT protocol tolerates f Byzantine faults with $n \leq 3f$ in partial synchrony.

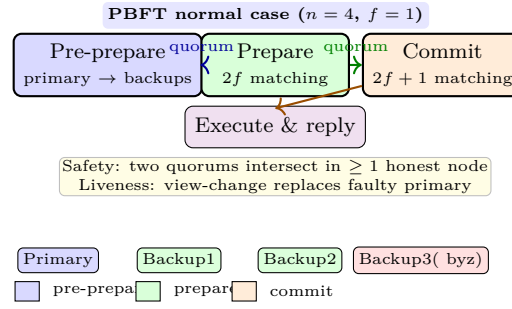


Figure 2.3: PBFT’s three-phase commit with quorum intersections. Each phase needs $2f + 1$ matching messages, guaranteeing any two quorums share an honest node – the source of the $3f + 1$ bound. Liveness comes from view-change timeouts.

Sketch. Partition $n = 3f$ into three groups A, B, C of f each. Byzantine C can mimic A to B and B to A , making A and B decide differently while each sees a quorum of $2f$. $\square$

In practice, Tendermint/Cosmos and HotStuff (used in later BFT chains) streamline PBFT’s view-change with a rotating leader and linear communication.

Example 2.1 (Choosing a consensus). A permissionless currency wants open entry and 10 min blocks: PoW or PoS fits. A supply-chain consortium with 20 known firms wanting 1 s finality: PBFT/Tendermint fits – faster, deterministic, but closed membership. Many modern chains hybridise: PoS for Sybil resistance plus a BFT finality gadget.

2.5 Difficulty Adjustment and Fork Choice

Difficulty does not stay fixed. Bitcoin adjusts every 2016 blocks: $D_{\text{new}} = D_{\text{old}} \cdot (2016 \cdot 600) / T_{\text{actual}}$ where T_{actual} is the time the last 2016 blocks took. If blocks came too fast ($T_{\text{actual}} < 2016 \cdot 600$), D rises; too slow, D falls. This negative feedback keeps block time near 10 min despite hash-rate swings.

Fork choice: when two blocks appear at the same height, honest nodes follow the *heaviest chain* (most accumulated work, which for equal D is the longest). An attacker who withholds blocks creates a hidden fork; releasing it can reorg the honest chain if it is heavier. PoS fork choice (LMD-GHOST plus Casper FFG) weighs attestations rather than work, but the heaviest-observed-subtree idea is analogous: the fork with most validator votes wins, and finalised checkpoints cannot be reverted without slashing.

Example 2.2 (Retarget arithmetic). Suppose last 2016 blocks took 12 days instead of 14. Then $D_{\text{new}} = D_{\text{old}} \cdot 14/12 \approx 1.17 D_{\text{old}}$: difficulty rises 17% to slow blocks back to 10 min. Halving hash rate has the opposite effect.

Remark 2.2 (Why 10 min?). Shorter block time reduces confirmation latency but raises fork rate (more simultaneous proposers due to propagation delay). Bitcoin’s 10 min is a compromise: propagation ($\sim 1\text{--}10\text{ s}$) $\ll$ block time, so forks are rare ($< 1\%$). Ethereum targets $\sim 12\text{ s}$ because GHOST/uncle rules tolerate more forks.

2.6 Exercises

- E2.1 **Block-time math.** Global hash rate $r = 200$ EH/s, difficulty $D = 6 \times 10^{13}$. Compute expected block time D/r and success probability per hash. If D doubles, what retarget keeps 10 min?
- E2.2 **PoW vs PoS cost.** An attacker rents 51% hash for 6 blocks vs bribes 51% stake with slashing risk. Model both costs with numbers and argue which is more “expensive to repeat.”
- E2.3 **PBFT trace.** With $n = 7$, $f = 2$, how many Prepare and Commit messages does an honest replica collect before proceeding? Show why $n = 6$ fails the intersection argument.
- E2.4 **Selfish mining.** A miner with 30% power withholds a found block. Sketch when releasing it causes a fork and when honest miners waste work. What threshold makes selfish mining profitable?
- E2.5 **Finality comparison.** A merchant accepts 0-conf vs 6-conf on PoW vs single-slot finality on PBFT. Assign reversion probabilities and justify your confirmation policy for a 10 k payment.

Chapter Notes

Nakamoto (2008) for PoW; Buterin’s Casper and the Ethereum PoS specs (2020-22) for PoS/slashing; Castro–Liskov, “Practical BFT” (OSDI 1999) for PBFT; Dwork–Lynch–Stockmeyer (1988) for partial synchrony; Yin et al., HotStuff (PODC 2019) for linear BFT. Narayanan et al., Ch. 2-3 is the best first-principles companion.

Chapter 3

Transactions, UTXO and the Account Model

“A transaction is not a row in a spreadsheet; it is a transformation of ownership that must conserve value and never double-spend.” — This chapter traces how value moves and why two competing models (UTXO vs accounts) exist.

From zero: no transaction background assumed. We start from a paper cheque (who pays whom, what prevents photocopying?) and build the digital equivalents. We define inputs/outputs, UTXO graphs and account nonces from scratch, then verify real transactions with Python.

Learning Outcomes

After this chapter you will be able to:

- (L1) define a transaction, input, output and script; explain conservation;
- (L2) construct and validate a UTXO graph and detect double-spends;
- (L3) contrast UTXO (Bitcoin) with account model (Ethereum) on state, parallelism and privacy;
- (L4) explain nonce, gas price, fee and replay protection (chain-id);
- (L5) simulate a mempool that orders by fee rate and packs a block.

3.1 Transactions as Value Flow

A *transaction* tx maps old coins to new coins. Each transaction has a list of *inputs* (references to coins being spent) and *outputs* (new coins being created), plus signatures proving ownership of the inputs.

Two rules must hold for validity:

- (i) *No double-spend*: each input must reference an unspent coin that exists and has not been spent before.
- (ii) *Conservation*: $\sum \text{inputs} \geq \sum \text{outputs}$; the difference is the fee (miner/validator reward).

If any rule breaks, the transaction is invalid regardless of signatures.

3.2 The UTXO Model

Bitcoin's *Unspent Transaction Output* model treats coins as discrete, consumable objects. The ledger state is the set $\mathcal{U}$ of UTXOs. Spending consumes UTXOs atomically and creates new ones.

Definition 3.1 (UTXO). A UTXO is a triple $(txid, index, value, scriptPubKey)$ where *scriptPubKey* is the spending condition (e.g., “pay to public key *pk*”). The set $\mathcal{U}$ contains exactly the UTXOs whose outputs have not been consumed as inputs elsewhere.

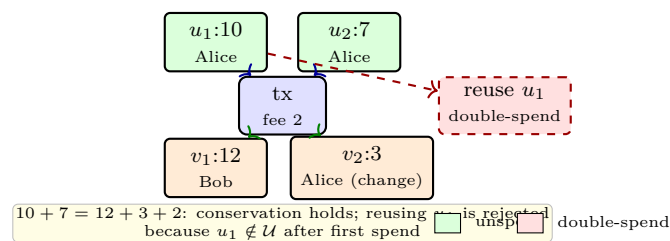


Figure 3.1: UTXO flow: Alice consumes u_1, u_2 ($10+7$) to pay Bob 12, reclaim 3 as change, and leave fee 2. Any attempt to reuse u_1 (red, dashed) fails because the UTXO set no longer contains it. The graph is a DAG.

UTXO is inherently parallelisable: disjoint input sets do not conflict. It also helps privacy (new addresses per output) but makes stateful contracts harder (no notion of “account balance”).

```

1  # UTXO validator (simplified)
2  utxo = {("tx0",0): {"value":10,"owner":"Alice"}, ("tx0",1): {"value":7,"owner":"Alice"}}
3  def validate(tx, utxo_set):
4      ins = tx["inputs"]      # list of (txid, idx)
5      outs = tx["outputs"]    # list of {"value":..., "owner":...}
6      # (i) existence + no double-spend within tx
7      assert len(set(ins)) == len(ins), "duplicate input in tx"
8      total_in = 0
9      for ref in ins:
10         assert ref in utxo_set, f"missing/already spent {ref}"
11         total_in += utxo_set[ref]["value"]
12     total_out = sum(o["value"] for o in outs)
13     assert total_in >= total_out, "creates money!"
14     fee = total_in - total_out
15     return fee
16
17 tx = {"inputs":[(("tx0",0),("tx0",1)), {"outputs":[{"value":12,"owner":"Bob"}, {"value":3,"owner":"Alice"}]}]
18 print("fee:", validate(tx, utxo)) # 2
19 # Double-spend attempt: reuse (tx0,0)
20 tx2 = {"inputs":[(("tx0",0)], "outputs":[{"value":9,"owner":"Carol"}]}
21 # Simulate applying tx first
22 del utxo[("tx0",0)]; del utxo[("tx0",1)]
  
```

```

23 | utxo[("tx1",0)] = {"value":12,"owner":"Bob"}; utxo[("tx1",1)] = {"value":3,"owner":"Alice"
    | }
24 | try: validate(tx2, utxo)
25 | except AssertionError as e: print("rejected:", e)

```

3.3 The Account Model

Ethereum uses *accounts*: each address maps to $(balance, nonce, codeHash, storageRoot)$. A transaction is a state transition

$$\sigma_{t+1} = \Upsilon(\sigma_t, tx), \quad tx = (nonce, gasPrice, gasLimit, to, value, data, v, r, s). \quad (3.1)$$

The *nonce* is a per-sender counter preventing replay; $gasPrice \times gasUsed$ pays for computation.

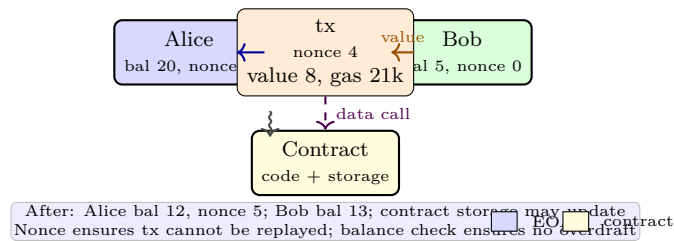


Figure 3.2: Account-model transition: a signed transaction increments the sender nonce, moves value, and may call contract code. The global state σ is a map address $\rightarrow$ account.

Example 3.1 (Replay without chain-id). Alice signs tx with nonce 4 on chain A. Without a chain-id in the signed data, an attacker can rebroadcast the same bytes on chain B where Alice also has funds and the same nonce is valid. EIP-155 fixes this by including *chainId* in the hash that is signed.

```

1 | # Account nonce + replay protection (simplified)
2 | state = {"Alice":{"balance":20,"nonce":4}, "Bob":{"balance":5,"nonce":0}}
3 | def apply_account_tx(state, tx, chain_id=1):
4 |     # tx: {from, to, value, nonce, gas_price, gas_limit, chain_id, sig_ok}
5 |     assert tx["sig_ok"], "bad signature"
6 |     assert tx["chain_id"] == chain_id, "wrong chain -- replay!"
7 |     s = state[tx["from"]]
8 |     assert tx["nonce"] == s["nonce"], f"bad nonce expected {s['nonce']}"
9 |     cost = tx["value"] + tx["gas_price"]*tx["gas_limit"]
10 |    assert s["balance"] >= cost, "insufficient funds"
11 |    # apply
12 |    s["balance"] -= cost; s["nonce"] += 1
13 |    state[tx["to"]]["balance"] += tx["value"]
14 |    return state
15 |
16 | tx = {"from":"Alice","to":"Bob","value":8,"nonce":4,"gas_price":0,"gas_limit":0,"chain_id":1,"sig_ok":True}
17 | print(apply_account_tx(state, tx))
18 | # Replaying same tx fails: nonce now 5
19 | try: apply_account_tx(state, tx)
20 | except AssertionError as e: print("replay rejected:", e)

```


3.4 Mempool, Fees and Block Packing

Users broadcast transactions to a *mempool* (pending set). Validators pack blocks greedily by *fee rate* (gasPrice or tip) to maximise revenue, subject to block gas limit.

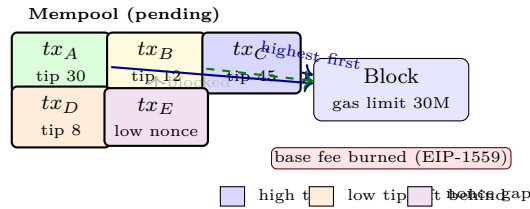


Figure 3.3: Mempool ordering by tip (priority fee). Validators greedily fill the block; transactions with nonce gaps or below the base fee wait. EIP-1559 burns the base fee and only the tip goes to the validator.

Remark 3.1 (Fee markets). EIP-1559 introduced a protocol-computed *base fee* that adjusts up/down by 12.5% per block to target half-full blocks; users add a *tip* to outbid others. This smooths fee spikes versus pure first-price auctions.

Comparison table:

	UTXO	Account
State	set $\mathcal{U}$	map $addr \rightarrow (bal, nonce, \dots)$
Parallelism	easy (disjoint inputs)	harder (overlaps on same account)
Smart contracts	limited (script)	rich (Turing-complete via EVM)
Privacy	better (fresh addr)	address reuse common

3.5 Exercises

- E3.1 **UTXO trace.** Given $\mathcal{U} = \{a:5, b:3, c:8\}$, trace a transaction consuming a, c to create $d:10$ (Bob) and change. What is the fee? Show the UTXO set before and after.
- E3.2 **Double-spend hunt.** Two transactions both consume $a:5$. Explain why only one can be included and how the mempool/consensus resolves the conflict. What if they are in competing forks?
- E3.3 **Account replay.** Alice has nonce 7. She signs tx with nonce 7 on chain 1. Show step-by-step what happens if the same bytes are submitted on chain 2 where her nonce is also 7 and chain-id is not checked.
- E3.4 **Fee estimator.** Write a Python function that, given a mempool list of $(gasUsed, tip)$ and a block gas limit, selects the revenue-maximising subset (knapsack) and compares it to greedy-by-tip order.
- E3.5 **UTXO vs account.** A game needs per-player state (level, inventory). Argue why UTXO is awkward and how the account model's storage trie helps. Propose a UTXO workaround and its cost.

Chapter Notes

Bitcoin’s transaction format in Nakamoto (2008) and BIP docs; UTXO set theory in Antonopoulos, *Mastering Bitcoin* (O’Reilly, 2nd ed.); Ethereum account model and EIP-1559 in Wood, *Yellow Paper* and ethereum.org specs; fee-market analysis in Roughgarden, “Transaction fee mechanism design for the Ethereum blockchain” (2021).

Chapter 4

Smart Contracts: Solidity and the EVM

“A smart contract is a program that lives on the chain, holds money, and runs exactly as written – whether the author regrets it afterwards or not.” — We learn to write, trace and defend such programs.

From zero: no EVM background assumed. We start from a vending machine (insert coin, get soda, no clerk) and turn it into code that lives on the blockchain. We define gas, storage, calldata and ABI from first principles before writing a line of Solidity.

Learning Outcomes

After this chapter you will be able to:

- (L1) write, compile and deploy a Solidity contract and call it via its ABI;
- (L2) trace EVM execution: stack, memory, storage, calldata and gas accounting;
- (L3) explain events, logs and the difference between `call` and `delegatecall`;
- (L4) identify and fix reentrancy, integer and access-control bugs;
- (L5) test with Hardhat/Foundry and estimate gas.

4.1 From Vending Machines to On-Chain Programs

A *smart contract* is deterministic code stored at an address. Anyone can send a transaction to that address with input data; every honest node re-executes the code and agrees on the outcome. The contract can hold ether/tokens, update persistent storage, and call other contracts. The key difference from a server: execution is *replicated and metered* – every node runs it, and every operation costs gas so infinite loops cannot halt the chain.

Two properties dominate design:

Determinism

No randomness, wall-clock or network I/O – all nodes must reach the same result from the same state and input.

Gas metering

Each EVM opcode has a gas price (e.g., `ADD`=3, `SSTORE`=20000). The transaction specifies a gas limit; exceeding it reverts.

4.2 The EVM Execution Model

The EVM is a stack machine with three data areas:

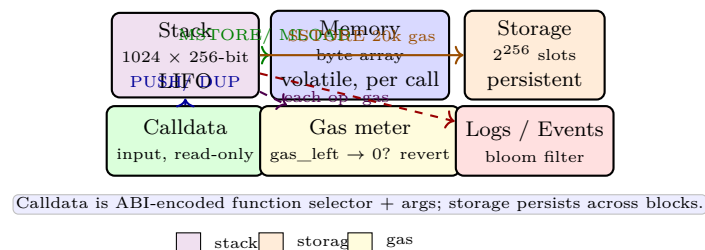


Figure 4.1: EVM machine model: a stack drives computation, memory is scratch space, storage is the durable key-value map, calldata carries the ABI-encoded call, and the gas meter halts runaway execution. Colour distinguishes volatile vs persistent areas.

Example 4.1 (ABI encoding). Calling `transfer(address,uint256)` with args (0xABC...,100) encodes as 4-byte selector 0xa9059cbb plus two 32-byte padded words. The EVM’s calldata is exactly those 68 bytes. Decoding is the callee’s first act.

4.2.1 Calls and delegatecall

- `addr.call(data)` runs callee code with callee storage, `msg.sender`=caller address.
- `addr.delegatecall(data)` runs callee code *with caller storage* – powerful for proxies but dangerous if mis-authorized.

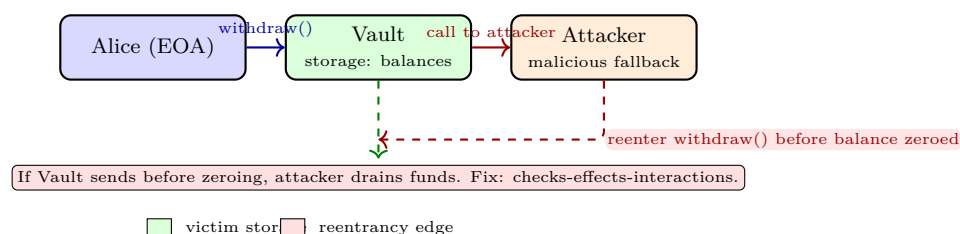


Figure 4.2: Reentrancy: the vault calls the attacker, which reenters the vault before the first call updates storage. Colour highlights victim vs attacker control.

4.3 Writing Solidity

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract SimpleVault {
5     mapping(address => uint256) public balances;
6     event Deposit(address indexed who, uint256 amount);
7     event Withdraw(address indexed who, uint256 amount);
8
9     function deposit() external payable {
10         balances[msg.sender] += msg.value;
11         emit Deposit(msg.sender, msg.value);
12     }
13     // VULNERABLE version -- for teaching (do NOT deploy)
14     function withdrawVuln() external {
15         uint256 bal = balances[msg.sender];
16         require(bal > 0, "empty");
17         (bool ok, ) = msg.sender.call{value: bal}(""); // external call first
18         require(ok, "send fail");
19         balances[msg.sender] = 0; // too late -- reentrancy!
20     }
21     // FIXED: checks-effects-interactions + guard
22     bool private locked;
23     modifier noReentrancy(){ require(!locked, "locked"); locked=true; _; locked=false; }
24     function withdraw() external noReentrancy {
25         uint256 bal = balances[msg.sender];
26         require(bal > 0, "empty");
27         balances[msg.sender] = 0; // effects before interaction
28         (bool ok, ) = msg.sender.call{value: bal}("");
29         require(ok, "send fail");
30         emit Withdraw(msg.sender, bal);
31     }
32 }

```

Key patterns: *checks-effects-interactions* (update storage before external calls), *reentrancy guard*, *pull over push*, and *pausable/ownable* for emergencies.

```

1 # Gas estimator (teaching -- real costs from Yellow Paper Table G)
2 GAS = {"ADD":3, "MUL":5, "SSTORE_new":20000, "SSTORE_reset":2900, "SLOAD":2100,
3        "CALL":700, "MSTORE":3, "KECCAK":30}
4 def estimate(ops):
5     return sum(GAS[o] for o in ops)
6 print("withdraw trace cost:", estimate(["SLOAD", "CALL", "SSTORE_reset"]))
7 # -> 9700 gas sketch; real trace includes memory expansion etc.
8 # ABI encode helper
9 import hashlib
10 def selector(sig): return hashlib.sha3_256(sig.encode()).hexdigest()[:8]
11 # Note: use keccak (not sha3 NIST) in production; hashlib.sha3 is close for demo
12 print("transfer selector demo:", selector("transfer(address,uint256)"))

```

4.3.1 Errors that cost millions

Three patterns have drained hundreds of millions: (1) *reentrancy* above; (2) *unchecked call return* – low-level `call` returns false on failure instead of reverting, so ignoring the bool silently loses funds; (3) *access control* – missing `onlyOwner` lets anyone call `selfdestruct` or mint. Solidity 0.8’s checked arithmetic fixed a fourth class (overflow), but price-oracle manipulation (using a spot AMM price as truth) remains subtle. The fix for each is a small code change, which is why audit checklists are so valuable.

Example 4.2 (Unchecked call). `addr.call{value:1 ether}("")` without checking `ok` can fail (out-of-gas, recipient reverts) while the vault thinks it succeeded and zeroes the balance anyway. Always write `require(ok)` or use OpenZeppelin’s `Address.sendValue`.

Remark 4.1 (Testing philosophy). Tests should cover *unit* (single function), *integration* (contract-to-contract calls), and *invariant/fuzz* (random sequences must preserve e.g. total deposits = contract balance + total withdrawn). Foundry’s `forge test -fuzz` and Halmos symbolic execution automate the last class.

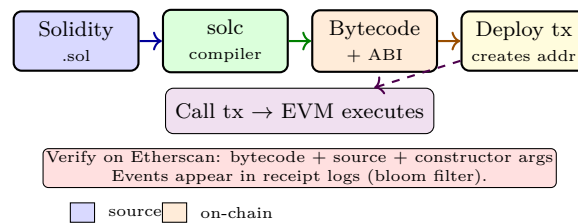


Figure 4.3: Contract lifecycle: source → bytecode/ABI → deploy transaction → on-chain address. Off-chain tools (Hardhat, Foundry) automate compile, test and verify steps.

4.4 Exercises

E4.1 **ABI drill.** Encode `transfer(0x1234...abcd, 1000)` by hand (selector + two words). Decode a given calldata hex back to function and args.

E4.2 **Reentrancy exploit.** Write an attacker contract that reenters `withdrawVuln` three times. Trace balances and call stack depth at each reentry. Then show why `withdraw` (fixed) blocks it.

E4.3 **Gas golf.** Given a loop that does `SSTORE` per iteration, estimate total gas for 100 iterations (new vs reset slots). Propose a packing that halves storage writes.

E4.4 **delegatecall danger.** A proxy uses `delegatecall` to a logic contract that has a `selfdestruct`. Explain how an attacker can destroy the proxy. What is the fix (UUPS vs transparent proxy)?

E4.5 **Test harness.** Write a Foundry/Hardhat test that deposits, withdraws and checks events for `SimpleVault`. Add a fuzz test that tries random deposit/withdraw sequences and asserts conservation.

Remark 4.2 (Upgradeability). Because contracts are immutable, teams use *proxy* patterns: a proxy holds storage and `delegatecalls` to a logic contract that can be swapped. The proxy itself holds the owner key, so securing that key and adding a timelock for upgrades is essential; otherwise the upgrade mechanism is the backdoor.

Example 4.3 (Events as cheap storage). Emitting `Deposit` costs ~375 gas plus 8 gas per byte, while storing the

same data in `SSTORE` costs 20 000 gas. For data that only needs to be observed off-chain (front-ends, indexers), events are 10–50x cheaper – one reason DeFi relies heavily on logs.

Remark 4.3 (Tooling ecosystem). Use OpenZeppelin contracts for audited primitives (ERC20, Ownable, ReentrancyGuard), Foundry for fast fuzzing, and Slither for static analysis. A typical CI pipeline runs `forge test`, `slither .`, and gas snapshots on every commit before deployment to testnet. Run fork tests against mainnet state to catch integration drift before mainnet deployment. Pin compiler versions and verify source on Etherscan so users can audit what they call.

Chapter Notes

Wood, *Ethereum Yellow Paper* (2014+) for EVM semantics and gas table G; Antonopoulos–Wood, *Mastering Ethereum* (O’Reilly) for Solidity patterns; Atzei–Bartoletti–Cimoli, “A survey of attacks on Ethereum smart contracts” (2017) for vulnerability taxonomy; Foundry book and Hardhat docs for toolchains; SWC Registry for weakness classification.

Chapter 5

Cryptography: Signatures, Merkle Trees and Commitments

“Cryptography replaces the clerk who checks your signature with mathematics that anyone can verify but no one can forge.” — This chapter rebuilds that mathematics from hashes and discrete logs to Merkle proofs and hiding commitments.

From zero: no crypto background assumed beyond Ch. 1 hashes. We start from wax seals (authenticate, tamper-evident, non-repudiable) and translate each property into a hash/signature/commitment primitive, with definitions, examples and Python verifiers.

Learning Outcomes

After this chapter you will be able to:

- (L1) define digital signatures (Gen/Sign/Verify) and state EUF-CMA security;
- (L2) derive Ethereum addresses from ECDSA/secp256k1 keys and verify a transaction signature;
- (L3) construct a Merkle tree, prove inclusion in $O(\log n)$ and verify the proof;
- (L4) explain hash commitments (hide + bind) and Pedersen-style intuition;
- (L5) sketch zero-knowledge proof intuition and where ZK is used in blockchains.

5.1 Digital Signatures: Seals that Mathematics Checks

A *digital signature scheme* is a triple (Gen, Sign, Verify):

- $\text{Gen}() \rightarrow (sk, pk)$ random keypair.
- $\text{Sign}(sk, m) \rightarrow \sigma$.

- $\text{Verify}(pk, m, \sigma) \in \{0, 1\}$.

Correctness: honest signatures verify. Security (EUF-CMA): without sk , no efficient adversary can forge a valid σ on a new m , even after seeing signatures on chosen messages.

Blockchains use ECDSA on curve secp256k1 (Bitcoin/Ethereum) and increasingly Ed25519/schnorr. Intuition: sk is a scalar d , $pk = dG$ (scalar multiplication of generator G); forging needs discrete log.

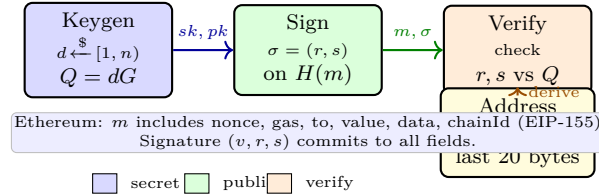


Figure 5.1: ECDSA lifecycle: key generation $\rightarrow$ signing the hash of $m \rightarrow$ verification with $pk \rightarrow$ address derivation. Only sk can produce a valid (r, s) , but anyone with pk can check.

```

1 # ECDSA sign/verify demo (uses ecdsa lib; pip install ecdsa)
2 import hashlib, ecdsa
3 sk = ecdsa.SigningKey.generate(curve=ecdsa.SECP256k1)
4 vk = sk.verifying_key
5 msg = b"send 10 coins to Bob, nonce=4, chain=1"
6 sig = sk.sign(msg, hashfunc=hashlib.sha256)
7 print("verify:", vk.verify(sig, msg, hashfunc=hashlib.sha256))
8 # Ethereum address = keccak(pubkey)[-20:]
9 pub_bytes = b"\x04" + vk.to_string() # uncompressed
10 # for demo, sha3 ~ keccak; use pysha3's keccak_256 in production
11 import hashlib
12 addr = hashlib.sha3_256(pub_bytes).hexdigest()[-40:]
13 print("addr 0x"+addr[:10]+"...")
14 # forge attempt with different key fails
15 sk2 = ecdsa.SigningKey.generate(curve=ecdsa.SECP256k1)
16 try: print("forge:", sk2.verifying_key.verify(sig, msg, hashfunc=hashlib.sha256))
17 except ecdsa.BadSignatureError: print("forge rejected")

```

5.2 Merkle Trees: One Hash Commits to Many

A *Merkle tree* hashes pairs recursively: leaves are $H(tx_i)$, internal node $H(left \parallel right)$, root M commits to the whole set.

Definition 5.1 (Merkle proof). For $n = 2^k$ leaves, a proof for tx_i is the k sibling hashes on the path to the root. Verification recomputes the root from tx_i plus siblings and compares to M . Cost $O(\log n)$ hashes vs $O(n)$ to hash everything.

```

1 import hashlib
2 def H(x): return hashlib.sha256(x).hexdigest()
3 def merkle_root(leaves):
4     layer = [H(l.encode()) for l in leaves]
5     while len(layer) > 1:
6         nxt = []

```

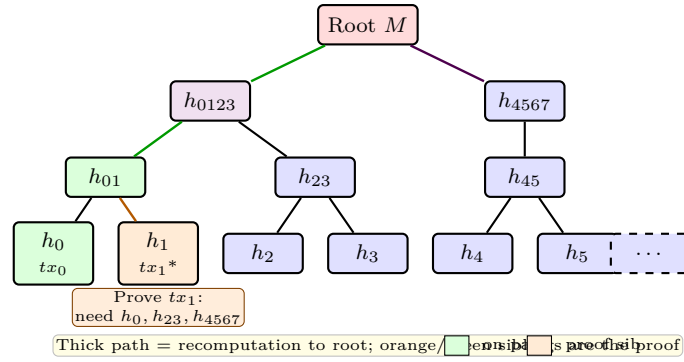


Figure 5.2: Merkle tree and inclusion proof: to prove tx_1 is in the block, reveal only 3 sibling hashes (not all n leaves); verifier recomputes the thick path to the root M stored in the header.

```

7     for i in range(0, len(layer), 2):
8         a, b = layer[i], layer[i+1] if i+1 < len(layer) else layer[i]
9         nxt.append(H((a+b).encode()))
10    layer = nxt
11    return layer[0]
12 def merkle_proof(leaves, idx):
13     # return sibling hashes along path
14     layer = [H(l.encode()) for l in leaves]
15     proof = []
16     i = idx
17     while len(layer) > 1:
18         sib = i ^ 1
19         if sib < len(layer): proof.append(layer[sib])
20         # build next layer
21         nxt=[]
22         for j in range(0, len(layer), 2):
23             a, b = layer[j], layer[j+1] if j+1 < len(layer) else layer[j]
24             nxt.append(H((a+b).encode()))
25         layer=nxt; i//=2
26     return proof
27
28 leaves = [f"tx{i}" for i in range(8)]
29 root = merkle_root(leaves)
30 print("root", root[:12]+"...", "proof for tx1", [p[:8] for p in merkle_proof(leaves,1)])

```

5.3 Commitments and Zero-Knowledge Intuition

A *commitment* $C = \text{Commit}(m; r)$ hides m (hiding) and binds to it (cannot open to $m' \neq m$). Simplest: $C = H(m \parallel r)$ with random r ; open by revealing (m, r) . Blockchains use commitments for sealed-bid auctions, voting and light-client bridges. A fancier variant, Pedersen $C = g^m h^r$, is additively homomorphic.

Zero-knowledge proofs prove “I know w with $P(w) = 1$ ” without revealing w . Intuition: prover and verifier play a game where a cheating prover would fail a random challenge with high probability, while an honest prover always passes and leaks nothing beyond the statement’s truth. ZK-rollups (Ch. 6) use this to prove “this batch of 1000 transactions was executed correctly” with a single short proof.

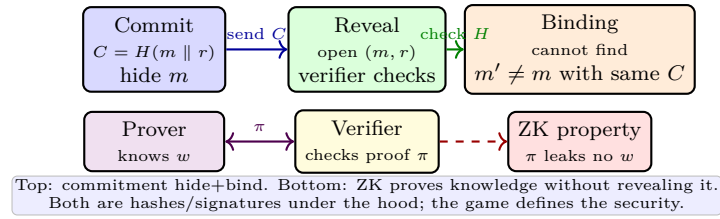


Figure 5.3: Commitment lifecycle (top) and ZK intuition (bottom). Commitments appear in sealed auctions and state channels; ZK proofs compress verification (Ch. 6).

Example 5.1 (Sealed-bid auction). Bidders commit $C_i = H(bid_i \parallel r_i)$ on-chain. After the deadline they reveal. No one can change bid_i after seeing others (binding) and no one learns bids before the reveal (hiding). A ZK variant can prove “my bid is highest” without revealing the amount.

Remark 5.1 (Hash-then-sign and domain separation). We never sign the raw message m (which may be huge); we sign $H(m)$. Collision resistance of H ensures signing $H(m)$ is as secure as signing m , while keeping signatures constant size. EIP-712 adds *domain separation* (chainId, contract address) into that hash so a signature for one dApp cannot be replayed in another – the same replay protection we saw in Ch. 3, now at the crypto layer.

Example 5.2 (Merkle in the wild). Bitcoin’s block header commits to all transactions via one Merkle root; Ethereum’s state uses a Merkle-Patricia trie so a single 32-byte state root commits to every account, storage slot and code hash. Light clients download only headers and request Merkle/MPT proofs for the keys they care about.

5.4 Exercises

- E5.1 **Sign/verify by hand.** Given toy parameters G, n small, compute **Sign** and **Verify** on a short message. Show that flipping one bit of the message fails verification.
- E5.2 **Merkle drill.** Build a tree for 8 leaves. Give the proof for leaf 5 (list siblings in order), recompute the root, and show what changes if leaf 2 is tampered.
- E5.3 **Light client.** A header stores root M . Explain step-by-step how a light client with only M verifies that tx is in the block using $O(\log n)$ hashes. Why is storing M enough?
- E5.4 **Commit game.** Alice commits $H(m \parallel r)$ with 128-bit r . Argue hiding against a brute-force adversary and binding against a collision finder. When does $H(m)$ alone fail to hide?
- E5.5 **ZK intuition.** State a statement you would ZK-prove in a rollup (e.g., “batch root is correct”). Who is prover/verifier, what is the witness, and why does the verifier not need to re-execute all transactions?

Chapter Notes

Katz–Lindell, Ch. 12–13 for signatures and hash commitments; Boneh–Shoup for pairing-based ZK; Nakamoto (2008) and Wood (Yellow Paper) for Merkle roots in headers; Ben-Sasson et al., “Zerocash” (2014) and Groth16 (2016) for the ZK lineage leading to ZK-rollups. Antonopoulos Ch. 4–6 is a practical companion for keys/addresses.

Chapter 6

Scaling: Channels, Rollups and Sharding

“You cannot have a single machine do all the world’s transactions without making it a centralised supercomputer. Scaling is about doing less on the bottleneck and proving it was done right.” — This chapter shows how.

From zero: no scaling background assumed. We start from a cafe tab (run a tab off-chain, settle once) and generalise to payment channels, rollups and sharded chains. Every construction is motivated by the trilemma, then formalised and traced with code.

Learning Outcomes

After this chapter you will be able to:

- (L1) state the scalability trilemma and why layer-1 throughput is bounded;
- (L2) trace a payment channel lifecycle (open, update, close, dispute) and explain HTLCs;
- (L3) contrast optimistic vs ZK rollups (fraud proofs vs validity proofs, data availability);
- (L4) sketch sharding, data-availability sampling and cross-shard communication;
- (L5) quantify fees and latency trade-offs across layer-2 options.

6.1 Why Layer-1 Cannot Scale Alone

A decentralised chain must keep verification cheap so anyone can run a node. Requiring every node to execute every transaction caps throughput (Bitcoin ≈ 7 tps, Ethereum ≈ 15 –30 tps before rollups). Raising the block size/gas limit raises throughput but raises node cost, reducing decentralisation – the *scalability trilemma* (decentralisation, security, scalability: pick two to maximise).

Scaling escapes by moving work *off* the bottleneck and posting only succinct commitments on-chain:

- *Channels*: two parties transact off-chain, settle net result on-chain.
- *Rollups*: a sequencer executes many layer-2 transactions and posts compressed data plus a proof on layer-1.
- *Sharding*: split state/processing across many chains that share security.

6.2 Payment Channels and Lightning

Two parties lock coins in a 2-of-2 multisig on-chain (funding tx). They then exchange signed off-chain states $S_i = (bal_A, bal_B, nonce = i)$ that spend the multisig with updated balances. Only the latest state matters; either party can close by posting it on-chain.

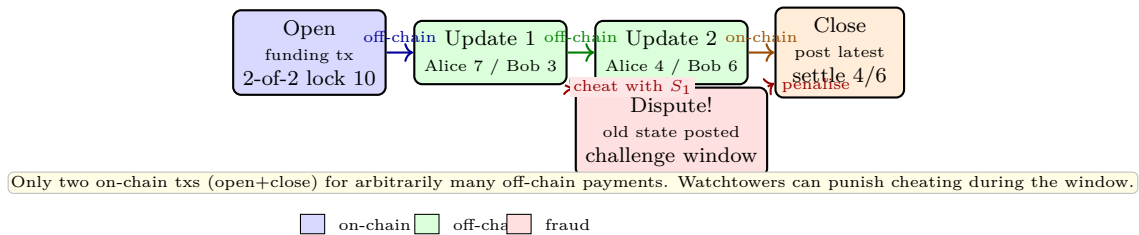


Figure 6.1: Payment channel lifecycle. The funding transaction and the closing transaction are the only on-chain footprints; everything else is instant, free, off-chain. Posting an outdated state can be challenged.

Hashed Timelock Contracts (HTLCs) chain channels into a network (Lightning): payment routes atomically across hops, each hop conditional on revealing a preimage r with $H(r) = h$ before timeout, so intermediaries cannot steal.

```

1  # Channel state updates (teaching)
2  class Channel:
3      def __init__(self, a_bal, b_bal):
4          self.states = [(a_bal, b_bal, 0)]
5          self.closed=False
6      def update(self, a_new, b_new):
7          assert not self.closed and a_new+b_new==10, "conservation + open"
8          self.states.append((a_new, b_new, len(self.states)))
9          return self.states[-1]
10     def close(self, idx=None):
11         # honest close uses latest; dispute may post old idx
12         s = self.states[idx] if idx is not None else self.states[-1]
13         self.closed=True
14         return s
15     def challenge(self, posted_idx):
16         # if someone posted old state, honest party can post newer
17         if posted_idx < len(self.states)-1:
18             return ("penalise cheat, reward honest", self.states[-1])
19         return ("close ok", self.states[posted_idx])
20
21 ch = Channel(5,5)
22 print(ch.update(7,3)); print(ch.update(4,6))
23 print("honest close", ch.close())
24 # Cheat trace
25 ch2 = Channel(5,5); ch2.update(7,3); ch2.update(4,6)
26 print("cheat with idx 1:", ch2.challenge(posted_idx=1))

```

Channels excel at many small payments between the same parties, but require liquidity lockup and watch-towers.

6.3 Rollups: Execute Off-Chain, Prove On-Chain

A rollup runs a layer-2 chain, batches many transactions, and posts to layer-1 either:

Optimistic

Post the new state root and assume correct; allow a *challenge window* (e.g., 7 days) where anyone can submit a *fraud proof* showing mis-execution, which reverts the batch.

ZK (validity)

Post the new state root plus a succinct ZK proof π that the transition was correct; layer-1 verifies π instantly (no window).

Both must post *data availability* (enough data for anyone to reconstruct layer-2 state) – otherwise a malicious sequencer could withhold data and freeze funds.

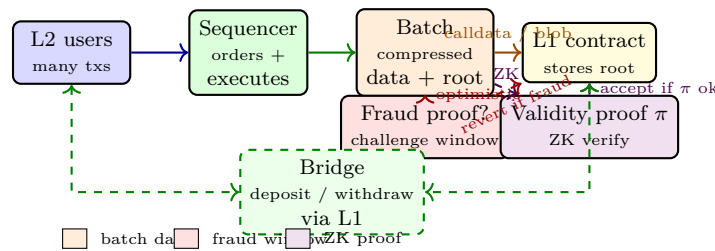


Figure 6.2: Rollup architecture: L2 batches are compressed and anchored on L1. Optimistic rollups rely on fraud proofs within a window; ZK rollups attach a validity proof. Both need a bridge and data availability.

```

1  # Rollup batch compression + fraud check (sketch)
2  import hashlib, json
3  def compress(txs): # naive: JSON -> bytes length
4      return json.dumps(txs).encode()
5  def state_root(state): return hashlib.sha256(json.dumps(state, sort_keys=True).encode()).hexdigest()
6  def apply_batch(state, txs):
7      for tx in txs: state[tx["to"]] = state.get(tx["to"], 0) + tx["value"]
8      return state
9  # Fraud proof: challenger shows a tx that was misapplied
10 def verify_fraud(state_before, txs, claimed_root):
11     correct = state_root(apply_batch(dict(state_before), txs))
12     return correct != claimed_root # True means fraud proven
13
14 s0={"Alice":100}
15 batch=[{"to":"Bob","value":10}, {"to":"Bob","value":5}]
16 print("compressed", len(compress(batch)), "bytes")
17 print("correct root", state_root(apply_batch(dict(s0), batch))[:12])
18 print("fraud?", verify_fraud(s0, batch, "00"*32))

```

Trade-offs: optimistic is simpler/EVM-equivalent but withdrawals are slow; ZK is instant but proving is expensive and general EVM ZK is harder.

6.4 Sharding and Data Availability

Sharding splits validators and state into k shards, each processing a subset of transactions. A beacon chain coordinates cross-shard commits. The hardest problem is *data availability*: how does a light client know a shard's block data was actually published without downloading it all? *Data-availability sampling* (DAS) uses erasure coding plus random spot-checks: if enough random samples are available, the full data is recoverable with high probability.

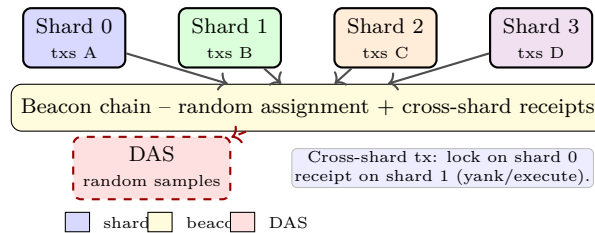


Figure 6.3: Sharding: execution parallelises across shards coordinated by a beacon chain. Data-availability sampling lets light clients gain confidence without full download. Colour separates per-shard load from coordination.

Ethereum's post-merge direction is *danksharding* with blobs (EIP-4844): layer-1 provides cheap blob space for rollup data, sharded across validators, rather than full state sharding.

Remark 6.1 (Fees and latency across layers). Channels: instant, near-zero fee, but only between the two participants and require inbound liquidity. Rollups: inherit L1 security, fees are L1 data cost amortised over many L2 txs (often 10–100x cheaper), latency is L2 block time (optimistic withdrawals 7 days, ZK near-instant). Sharding: throughput multiplies with shard count but cross-shard latency adds a round-trip via the beacon. Choosing among them is a latency vs cost vs generality trade-off.

Example 6.1 (Lightning routing). Alice wants to pay Dave but has no direct channel. She routes via Bob and Carol using HTLCs: each hop locks coins contingent on knowledge of r with $H(r) = h$. Dave, who knows r , claims first, revealing r that propagates backwards atomically. If any hop times out, all HTLCs refund – no intermediary can steal.

Example 6.2 (Blob economics). EIP-4844 introduces blobs that are not executed by L1, only stored for ~ 18 days and priced separately (blob gas). A rollup posting 120 KB per batch used to pay $\approx 1.9\text{M}$ gas in calldata; with blobs the same data costs $\approx 150\text{k}$ blob gas – an order-of-magnitude saving that makes L2 fees sustainable.

6.5 Exercises

E6.1 **Channel math.** Alice and Bob open with 10 coins each. After 100 off-chain payments net is Alice 12 / Bob 8. How many on-chain txs occurred? What happens if Bob posts an old state where he had 15?

E6.2 **HTLC route.** Three hops Alice->Bob->Carol->Dave with hash $h = H(r)$. Sketch the lock/unlock order and prove atomicity: Dave reveals r to claim, which lets Carol claim, etc., or all time out.

- E6.3 **Rollup costs.** A batch of 1000 L2 txs compresses 10x. If L1 calldata costs 16 gas/byte, estimate gas saved vs posting each tx on L1. Repeat for blobs at 1 gas/byte.
- E6.4 **Optimistic vs ZK.** A new game needs instant withdrawals and full EVM compatibility. Recommend optimistic or ZK and justify with fraud-window vs prover-cost trade-offs.
- E6.5 **DAS intuition.** A block is erasure-coded into 100 chunks, any 50 recover it. A light client samples 10 random chunks, all available. Bound the probability that $> 50\%$ of data is actually withheld.

Chapter Notes

Poon–Dryja, Lightning Network (2016) for channels/HTLCs; Buterin, “A rollup-centric Ethereum roadmap” (2020) and EIP-4844 for blobs; Gudgeon et al., SoK on layer-2 (2020); Al-Bassam et al., “Fraud and data availability proofs” (2019) for DAS; Ethereum danksharding specs (ethereum.org).

Chapter 7

Security and Attacks: Double-Spend, MEV and Verification

“A blockchain is only as secure as its cheapest attack.” — We enumerate the cheapest attacks, quantify them, and show how formal methods turn “looks safe” into “proved safe.”

From zero: no attack background assumed. We start from a shop that accepts 0-confirmation payments and ask how you can get the goods and keep the coins. We define adversary models, trace forks and MEV, and end with machine-checked proofs.

Learning Outcomes

After this chapter you will be able to:

- (L1) model double-spend / 51% / eclipse attacks and compute confirmation safety;
- (L2) explain MEV, front-running and proposer-builder separation (PBS);
- (L3) trace long-range and nothing-at-stake attacks on PoS/BFT;
- (L4) apply a smart-contract audit checklist (reentrancy, access, oracle, random);
- (L5) sketch formal verification: invariants, model checking and symbolic execution.

7.1 Double-Spend and Fork Races

In PoW, an attacker who secretly mines a fork can try to *double-spend*: pay a merchant on the honest chain, receive goods, then publish a longer attacker fork that excludes the payment and reclaims the coins. The attack succeeds if the attacker fork overtakes the honest fork before the merchant’s confirmation window k passes.

Nakamoto modelled block arrivals as Poisson: if attacker has fraction q of hash power ($p = 1 - q$ honest), the probability to ever catch up from z blocks behind is $(q/p)^z$ when $q < p$, else 1. So waiting $k = 6$ with $q = 0.3$


```

1 # Sandwich detector (toy AMM, constant product x*y=k)
2 def swap(x,y, dx): # user swaps dx of X for Y
3     k=x*y
4     new_x=x+dx
5     new_y=k/new_x
6     return y-new_y, new_x, new_y # dy, new reserves
7 x,y=10000, 10000
8 # Honest large swap
9 dy,_,_=swap(x,y,1000); print("user gets", round(dy,1), "without sandwich")
10 # Sandwich: searcher front-runs with 500, then user, then searcher sells
11 dy1,x1,y1=swap(x,y,500)
12 dyU,x2,y2=swap(x1,y1,1000)
13 # searcher sells what they bought (500 worth of X equivalent)
14 # simplified: searcher had bought dy1 Y, now swaps back
15 profit = 500 - (x*y/y1 - x) # approximate
16 print(f"front-run buy Y={round(dy1,1)}, user now gets worse price; searcher profit ~ {
    round(dy1 - 480,1)}")

```

7.3 Auditing and Formal Verification

Audits follow a checklist: reentrancy, access control (`onlyOwner`), integer over/underflow (pre-0.8), oracle manipulation, unchecked `call`, front-running, and denial-of-service. Tools help but do not replace reasoning.

Formal verification proves invariants hold for *all* executions: model the contract as a state machine, write a spec (e.g., “totalSupply is conserved, no one can withdraw more than deposited”), and let a solver exhaust the state space.

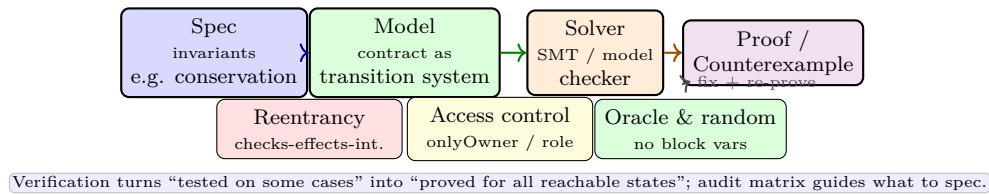


Figure 7.3: Verification pipeline (top) and audit focus areas (bottom). Tools like Certora, Halmos, and Mythril implement the solver step; the spec step is human judgement.

Example 7.1 (Invariant for SimpleVault). Invariant I : $\sum_{addr} balances[addr] \leq \text{contract balance}$ always holds, and $balances[msg.sender]$ only decreases by *withdraw* by *msg.sender*. A model checker flags *withdrawVuln* (Ch. 4) because between the call and zeroing, I is violated via reentry.

Remark 7.1 (Defense in depth). No single layer suffices: consensus defends against rewrites, networking defends against eclipse, contract code defends against reentrancy, and formal specs defend against logic bugs. The cheapest attack is the one you forgot to model, so threat-model every interface (user input, oracle, cross-chain bridge, governance).

Example 7.2 (Long-range attack). In PoS, old validators who have withdrawn can sell their old keys. An attacker buying those keys can forge an alternative history from the distant past at no cost. Clients that have been offline must not sync from genesis; they need a recent *weak subjectivity checkpoint* from a trusted source (friends, block explorer, or the protocol’s baked checkpoint) to know which fork is canonical. This is why PoS needs social consensus in addition to protocol rules.

Remark 7.2 (Bridge risk). Cross-chain bridges lock coins on chain A and mint wrapped coins on chain B. The bridge’s validator set or multisig becomes a single point of failure: compromising it lets an attacker mint unbacked wrapped coins. Over \$2B has been lost this way (2022 alone), more than any single contract bug class.

Example 7.3 (Selfish mining payoff). A selfish miner with 30% power finds a block and withholds it. If the honest network finds the next block, the selfish miner races: if their private chain is still ahead they publish and win, otherwise they lose the withheld work. Profitability exceeds honest mining only above $\approx 33\%$ (Eyal–Sirer), so 30% alone is not enough without network advantage.

Remark 7.3 (Eclipse vs Sybil). Eclipse and Sybil are complementary: Sybil creates many identities cheaply; eclipse uses them to monopolise a victim’s peer table. Defenses: diversify peer selection by IP prefix, ASN and latency, require proof-of-work for peer admission, and query multiple independent explorers for chain tips.

Remark 7.4 (Responsible disclosure). When you find a bridge or contract bug, follow responsible disclosure: notify the team privately, give a fix window, then publish. Publicly exploiting or front-running the fix harms users who have not yet upgraded. Bug bounties (Immunefi) align incentives for white-hat disclosure.

Example 7.4 (Price oracle attack). A lending market uses the spot price from a single AMM pool as its oracle. An attacker flash-loans a huge amount, skews the pool price 10x within the same transaction, borrows against the inflated collateral, then restores the price – all atomically. Fix: use a time-weighted average (TWAP) or a decentralised oracle (Chainlink) that aggregates multiple sources and cannot be moved in one block.

Remark 7.5 (Audit as sampling). An audit is a sample, not a proof: auditors read code for a few weeks against a checklist, but cannot explore every path. Treat an audit like a code review, not a guarantee. Combine it with bug bounties, invariant tests and formal verification for layered assurance.

Example 7.5 (Checklist in action). Run through the audit matrix for a new DEX: (i) reentrancy on callbacks, (ii) access on `setFees`, (iii) oracle freshness on `getPrice`, (iv) `tx.origin` misuse, (v) griefing where a user can block withdrawals. Each row takes minutes to check but saves millions.

7.4 Exercises

- E7.1 **Confirmation sizing.** With $q = 0.35$, compute catch-up probability for $z = 1, 3, 6, 12$. How many confirmations keep success $< 0.1\%$? What merchant policy does this imply for high-value settlement?
- E7.2 **Eclipse cost.** A victim connects to 8 random peers, attacker runs 200 Sybil nodes out of 5000 total. Estimate eclipse success if peer selection is uniform. How does peer diversity (IP, ASN) mitigate?
- E7.3 **Sandwich profit.** For constant-product AMM with reserves (x, y) , derive searcher profit as a function of front-run size and victim size. Plot the optimum front-run amount.
- E7.4 **Audit a vault.** Given a vault that uses `tx.origin` for auth and `block.timestamp` for random, list exploits and write fixes with code diffs.
- E7.5 **Invariant writing.** Write two invariants for an ERC-20 (e.g., totalSupply conservation, allowance monotonicity) in pseudo-Solidity/ spec language and explain how a symbolic executor would check them.

Chapter Notes

Nakamoto (2008) and Rosenfeld (2014) for double-spend math; Daian et al., “Flash Boys 2.0” (2019) for MEV; Buterin et al., PBS specs (2021-23); Atzei–Bartoletti–Cimoli (2017) for the attack survey; Certora/Halmos/Mythril docs and Bhargavan et al., “Formal verification of smart contracts” (2016) for machine-checked proofs.

Chapter 8

Applications, Tokenomics and Governance

“A blockchain alone does nothing; it becomes useful when contracts encode an application and tokens encode incentives.” — This final chapter maps patterns and economics onto the machinery of Ch. 1–7.

From zero: no DeFi/tokenomics background assumed. We start from a farmers’ market (traders, lenders, insurers, voting co-ops) and translate each role into a contract primitive, a token design and a governance rule. Every formula is motivated before formalised.

Learning Outcomes

After this chapter you will be able to:

- (L1) map DeFi/NFT/DAO/supply-chain use cases to contract primitives (AMM, lending, oracle, vault);
- (L2) design tokenomics: supply schedule, inflation, staking yield and bonding curves;
- (L3) explain governance (on-chain vote, timelock, delegation) and its attack surface;
- (L4) critique decentralisation trade-offs and regulatory tensions (privacy vs compliance);
- (L5) evaluate a whitepaper’s claims against the verification and scaling tools of Ch. 5–7.

8.1 Application Patterns

Most blockchain apps compose a small set of primitives:

Example 8.1 (Constant-product AMM). Reserves (x, y) with invariant $x \cdot y = k$. Swapping Δx for Δy gives $\Delta y = y - k/(x + \Delta x)$. Price impact grows with trade size; arbitrageurs restore x/y to the external market price, paying fees to liquidity providers. Impermanent loss is the opportunity cost vs holding.

Other domains: *supply chain* (provenance via tokenised batches plus ZK for privacy), *identity* (verifiable credentials anchored on-chain), and *gaming* (true ownership of in-game assets via NFTs plus on-chain logic).

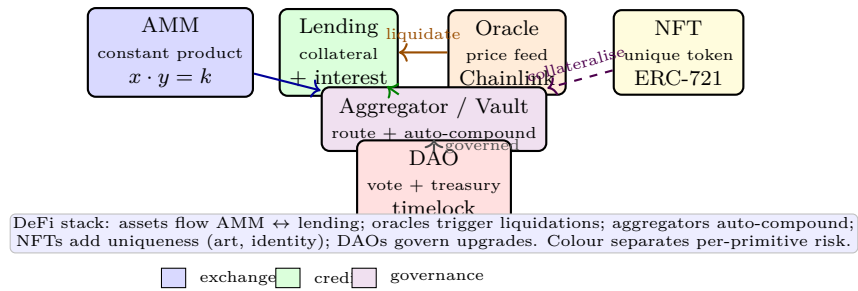


Figure 8.1: Composable primitives: AMM for swapping, lending for leverage, oracles for external truth, NFTs for uniqueness, and DAOs for governance. Real apps (Uniswap, Aave, MakerDAO) are compositions, not singletons.

8.2 Tokenomics: Designing Incentives

A *token* is a programmable unit of value/utility/governance. Good tokenomics answers: supply schedule, allocation, inflation, yield and value accrual.

Definition 8.1 (Supply and emission). *Max supply* $S_{\max}$ is the cap; *circulating* S_t at time t follows an *emission curve* (e.g., Bitcoin halving: reward halves every 210k blocks). *Inflation* $i_t = (S_{t+1} - S_t)/S_t$ funds security (staking rewards) but dilutes holders. *Staking yield* y_t pays stakers to lock capital; real yield = $y_t - i_t$ if all stake.

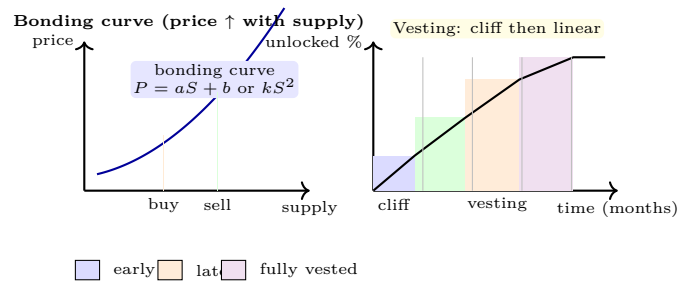


Figure 8.2: Left: bonding curve (price as function of supply) for continuous token offerings. Right: vesting schedule (cliff + linear) to align long-term incentives. Colour tracks time/price progression.

A *bonding curve* $P(S)$ sets price by supply (e.g., $P = aS$), enabling permissionless issuance. *Vesting* (cliff then linear unlock) prevents early dumps. Key tensions: high inflation secures the chain but dilutes; deflationary burns (EIP-1559) help holders but may underpay security.

```

1  # AMM constant-product swap + bonding curve pricer
2  def amm_swap(x,y, dx):
3      k=x*y
4      dy = y - k/(x+dx)
5      return dy, x+dx, y-dy
6
7  x,y=100000, 100000 # reserves
8  for sz in [100,1000,5000,10000]:
9      dy,_,_=amm_swap(x,y, sz)
10     price = dy/sz
11     print(f"swap {sz}: get {round(dy,1)}, avg price {round(price,4)} (slippage up)")
12 # Bonding curve linear: P = 0.01*S + 1, cost to mint dS = integral P dS
13 def bonding_cost(S, dS, a=0.01, b=1.0):

```

```

14     # integral of a*s+b ds from S to S+dS
15     return 0.5*a*((S+dS)**2 - S**2) + b*dS
16 print("mint 100 at S=1000:", round(bonding_cost(1000,100),2))
17 # Vesting
18 def vested(t, cliff=6, total=24):
19     if t < cliff: return 0.0
20     return min(1.0, (t-cliff)/(total-cliff))
21 for m in [0,6,12,18,24]:
22     print(f"month {m}: {vested(m)*100:.0f}% unlocked")

```

8.3 Governance, DAOs and Trade-offs

Code needs upgrades and treasuries need spending; *governance* decides how. On-chain governance token vote → timelock → execute is the DAO pattern, but it faces plutocracy (whales dominate), voter apathy, and vote-buying/bribery (MEV of governance).

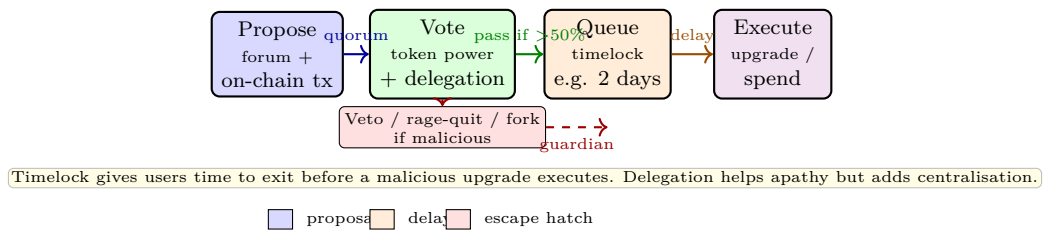


Figure 8.3: DAO governance lifecycle: proposal → vote (with delegation) → timelocked queue → execution. Red shows escape hatches (veto, rage-quit, fork) when governance is captured.

Remark 8.1 (Decentralisation spectrum). “Decentralised” is not binary: count clients, operators, geographic spread, and governance concentration. A chain with one dominant client, three validators and whale-controlled governance is technically a blockchain but not meaningfully decentralised. Regulation adds another axis: privacy (ZK, mixers) vs compliance (KYC at on/off-ramps, travel rule).

Example 8.2 (So you want to build a token?). Checklist before writing code: (1) what is scarce and why should it be on-chain rather than in a database? (2) who must hold the token for the system to function (users, stakers, LPs)? (3) what sinks/burns offset emissions? (4) how does governance upgrade the system without capture? If you cannot answer all four, you may not need a token – a common lesson in the 2022 crash.

Remark 8.2 (Regulation and the road ahead). Regulators increasingly treat tokens as securities, stablecoins as e-money, and mixers as money-transmission. Designing for compliance (allow-listed pools, view keys, audit trails) versus maximal privacy is now a product decision with legal consequences. The technology of Ch. 1–7 is neutral; the deployment is not.

Example 8.3 (Reading a whitepaper). When assessing a new chain, ask: (1) what is the Sybil-resistance mechanism and its cost to attack? (2) what is the light-client proof size? (3) where is data availability posted and at what cost? (4) what is the formal spec for the token’s monetary policy? Vague answers to any of these are red flags.

```

1 # DAO voting + timelock simulator
2 from collections import defaultdict

```



```

3 votes = defaultdict(int) # choice -> power
4 # delegation: Alice delegates to Bob
5 deleg = {"Alice": "Bob"}
6 power = {"Bob": 60, "Carol": 40, "Dave": 15} # Bob includes Alice's 10
7 # tally
8 for voter, p in power.items():
9     votes["yes"] += p if voter in ["Bob", "Carol"] else 0
10    votes["no"] += p if voter=="Dave" else 0
11 print("tally", dict(votes), "pass?" , votes["yes"] > votes["no"])
12 # timelock: proposal queued at t=0, executable after delay
13 delay=2 # days
14 for t in [0,1,2,3]:
15     print(f"day {t}: {'executable' if t>=delay else 'timelocked'}")
16 # Bonding outcome after DAO spend
17 treasury=1_000_000
18 spend=200_000
19 print("treasury after spend:", treasury-spend)

```

8.4 Exercises

- E8.1 **AMM slippage.** Reserves $(100k, 100k)$, $k = 10^{10}$. Compute output and average price for swaps $1k, 10k, 50k$. Plot slippage vs size. At what size does price deviate 5% from spot?
- E8.2 **Emission design.** Design a 4-year halving schedule from 50 to 3.125 per block for 10 min blocks. Compute total supply after 4 halvings and annual inflation each year.
- E8.3 **Vesting game.** Team gets 20% with 1-year cliff + 3-year linear vest. Investor gets 10% with 6-month cliff + 1-year linear. Sketch both curves and argue who can dump first and the market impact.
- E8.4 **Governance attack.** An attacker flash-loans governance tokens, votes, and repays in one block. Explain why snapshot-at-proposal-start and timelock mitigate this. Propose a vote-escrow (veToken) alternative.
- E8.5 **Whitepaper critique.** Pick any live project's whitepaper claims (throughput, decentralisation, tokenomics). Map each claim to a Ch. 2/5/6/7 check (consensus proof, Merkle cost, rollup data, MEV) and flag one unsubstantiated statement.

Chapter Notes

Buterin (2013) whitepaper and Wood (Yellow Paper) for app/token foundations; Adams et al., Uniswap v2/v3 for AMMs; EIP-1559 and tokenomics surveys (Messari, 2022) for supply design; DAO governance and bribery in Daian–Goldfeder analyses and Optimism Collective docs. Narayanan et al., Ch. 9–10 for the application landscape; Bonneau et al., SoK on cryptocurrencies (2015) for the research map.